

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE:CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

Coding Challenge

Code of Conduct:

I Gurusurya G 192311332(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Gurusurya
Signature of the student:

Annexure A

Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.
5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

1. IPSec Authentication Header (AH) Simulation

Concept

The Authentication Header (RFC 4302) provides connectionless data-integrity and data-origin authentication for IP packets, and optional anti-replay protection. AH does NOT provide confidentiality — the payload remains in plaintext. It works by computing an Integrity Check Value (ICV), a keyed hash (HMAC) over the immutable header fields and the payload, using a secret key that is part of a negotiated Security Association. The receiver recomputes the ICV and compares it to the one received; any mismatch means the packet was modified or is not authentic.

Design

- IPpacket: holds source/destination IP, protocol, and payload; exposes only the fields that AH covers (mutable fields such as TTL are excluded, as in the real RFC).
- AuthenticationHeader: holds the SPI and shared secret key from the SA; generate() computes and attaches an HMAC-SHA256 ICV with a monotonically increasing sequence number (anti-replay); verify() recomputes the ICV using hmac.compare_digest for constant-time comparison.
- The demo shows three cases: (a) an unmodified packet passing verification, (b) a tampered payload being rejected, and (c) a spoofed source IP being rejected — since the source address is covered by the ICV.

Python Implementation

```
"""
1. IPSec Authentication Header (AH) Simulation
-----
AH provides connectionless integrity, data origin authentication and
(optional) anti-replay protection for IP packets. It does NOT encrypt
the payload -- it only appends an Integrity Check Value (ICV), which in
this simulation is an HMAC computed over the (mostly) immutable fields
of the IP header plus the payload.

This program:
* builds a simplified IP packet (header dict + payload bytes)
* computes an HMAC-SHA256 ICV using a shared secret key (the AH "key"
  negotiated by a Security Association)
* attaches the ICV as the Authentication Header
* simulates transmission and verifies the ICV at the receiver
* demonstrates detection of tampering (integrity failure)
"""

import hmac
import hashlib
import json
import time

class IPPacket:
    """A very small stand-in for an IPv4 packet."""

    def __init__(self, src_ip, dst_ip, protocol, payload: bytes):
        self.src_ip = src_ip
        self.dst_ip = dst_ip
        self.protocol = protocol
        self.payload = payload

    def immutable_header_bytes(self):
        """
        Only the fields that do NOT change in transit (src/dst address,
        protocol) are covered by AH, mirroring RFC 4302 which excludes
        mutable fields such as TTL/Hop Limit from the ICV calculation.
        """
        header = {
            "src_ip": self.src_ip,
            "dst_ip": self.dst_ip,
            "protocol": self.protocol,
        }
        return json.dumps(header, sort_keys=True).encode()

    def __repr__(self):
        return f"IPPacket(src={self.src_ip}, dst={self.dst_ip}, proto={self.protocol}, payload={self.payload})"

class AuthenticationHeader:
    """Implements RFC 4302 style AH processing."""

    def __init__(self, spi: int, key: bytes, hash_algo="sha256"):
        self.spi = spi # Security Parameter Index (identifies the SA)
        self.key = key # shared secret key from the SA
        self.hash_algo = hash_algo
        self.seq_num = 0 # anti-replay sequence counter

    def _compute_icv(self, packet: IPPacket, seq_num: int) -> str:
        mac = hmac.new(self.key, digestmod=self.hash_algo)
        mac.update(self.spi.to_bytes(4, "big"))
        mac.update(seq_num.to_bytes(4, "big"))
        mac.update(packet.immutable_header_bytes())
        mac.update(packet.payload)
        return mac.hexdigest()

    def generate(self, packet: IPPacket) -> dict:
        """Sender side: attach AH (SPI, sequence number, ICV) to the packet."""
        self.seq_num += 1
        icv = self._compute_icv(packet, self.seq_num)
        ah = {
```

```

        "spi": self.spi,
        "seq_num": self.seq_num,
        "icv": icv,
        "timestamp": time.time(),
    }
    return {"header": packet, "ah": ah, "payload": packet.payload}

def verify(self, transmitted: dict) -> bool:
    """Receiver side: recompute the ICV and compare (constant-time)."""
    packet = transmitted["header"]
    ah = transmitted["ah"]
    expected_icv = self._compute_icv(packet, ah["seq_num"])
    return hmac.compare_digest(expected_icv, ah["icv"])

def demo():
    print("=" * 70)
    print("IPSec AH (Authentication Header) Simulation")
    print("=" * 70)

    shared_key = hashlib.sha256(b"pre-shared-ipsec-key").digest()
    ah = AuthenticationHeader(spi=0x1001, key=shared_key)

    packet = IPPacket("192.168.1.10", "192.168.1.20", "TCP", b"Hello, this is a secure IP payload!")
    print(f"\nOriginal packet: {packet}")

    transmitted = ah.generate(packet)
    print(f"AH generated -> SPI={hex(transmitted['ah']['spi'])}, "
          f"SeqNum={transmitted['ah']['seq_num']}, ICV={transmitted['ah']['icv'][:32]}...")

    # --- Case 1: packet arrives unmodified ---
    print("\n[Case 1] Packet delivered without tampering:")
    print(" Integrity check:", "PASSED" if ah.verify(transmitted) else "FAILED")

    # --- Case 2: an attacker tampers with the payload in transit ---
    print("\n[Case 2] Packet payload tampered with in transit:")
    tampered = {
        "header": IPPacket(packet.src_ip, packet.dst_ip, packet.protocol, b"Hello, this is a HACKED
payload!!"),
        "ah": transmitted["ah"],
        "payload": b"Hello, this is a HACKED payload!!",
    }
    print(" Integrity check:", "PASSED" if ah.verify(tampered) else "FAILED (tampering detected)")

    # --- Case 3: an attacker spoofs the source IP (header field covered by AH) ---
    print("\n[Case 3] Source IP spoofed (header field protected by AH):")
    spoofed = {
        "header": IPPacket("10.0.0.99", packet.dst_ip, packet.protocol, packet.payload),
        "ah": transmitted["ah"],
        "payload": packet.payload,
    }
    print(" Integrity check:", "PASSED" if ah.verify(spoofed) else "FAILED (spoofing detected)")

if __name__ == "__main__":
    demo()

```

Sample Output

```

=====
IPSec AH (Authentication Header) Simulation
=====

Original packet: IPPacket(src=192.168.1.10, dst=192.168.1.20, proto=TCP, payload=b'Hello, this is a secure IP
payload!')
AH generated -> SPI=0x1001, SeqNum=1, ICV=12d2f30f60656796dce4e456cad095d...

[Case 1] Packet delivered without tampering:
 Integrity check: PASSED

[Case 2] Packet payload tampered with in transit:

```

Integrity check: FAILED (tampering detected)

[Case 3] Source IP spoofed (header field protected by AH):

Integrity check: FAILED (spoofing detected)

2. IPSec Encapsulating Security Payload (ESP) Simulation

Concept

The Encapsulating Security Payload (RFC 4303) protocol provides confidentiality by encrypting the IP payload, and can optionally provide integrity/authentication as well. This program implements ESP in an Encrypt-then-MAC combined mode: the payload is padded and encrypted with AES-256 in CBC mode using a fresh random Initialization Vector (IV) for every packet, and an HMAC-SHA256 is then computed over the SPI, sequence number, IV, and ciphertext to protect integrity and detect tampering.

Design

- `ESP.encrypt()`: pads the plaintext (PKCS7), generates a random IV, encrypts with AES-256-CBC using the SA's encryption key, then computes an HMAC (ICV) over the resulting fields using the SA's authentication key.
- `ESP.decrypt()`: first verifies the ICV (rejecting the packet immediately on any mismatch, before ever attempting decryption — a defense against padding-oracle style attacks), then decrypts and un-pads the plaintext.
- The demo shows successful round-trip decryption, detection of a single flipped ciphertext bit (integrity failure), and confirms that an eavesdropper without the key only observes opaque ciphertext on the wire.

Python Implementation

```
"""
2. Encapsulating Security Payload (ESP) Simulation
-----
ESP (RFC 4303) provides confidentiality (encryption), and optionally
integrity/authentication, replay protection, for the IP payload.
Unlike AH, ESP encrypts the payload, so an eavesdropper cannot read it.

This program implements ESP in "combined mode" using AES-256 in
CBC mode for confidentiality plus an HMAC-SHA256 for integrity
(Encrypt-then-MAC), which mirrors how real ESP transforms such as
AES-CBC + HMAC-SHA are combined in an SA.
"""

import os
import hmac
import hashlib
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding

class ESP:
    def __init__(self, spi: int, enc_key: bytes, auth_key: bytes):
        self.spi = spi
        self.enc_key = enc_key      # AES-256 key (32 bytes)
        self.auth_key = auth_key   # HMAC key
        self.seq_num = 0

    # ----- Sender side -----
    def encrypt(self, plaintext_payload: bytes) -> dict:
        self.seq_num += 1
        iv = os.urandom(16)

        padder = padding.PKCS7(algorithms.AES.block_size).padder()
        padded = padder.update(plaintext_payload) + padder.finalize()

        cipher = Cipher(algorithms.AES(self.enc_key), modes.CBC(iv))
        encryptor = cipher.encryptor()
        ciphertext = encryptor.update(padded) + encryptor.finalize()

        esp_packet = {
            "spi": self.spi,
```

```

        "seq_num": self.seq_num,
        "iv": iv,
        "ciphertext": ciphertext,
    }
    # Encrypt-then-MAC: ICV covers SPI, seq, IV and ciphertext
    esp_packet["icv"] = self._icv(esp_packet)
    return esp_packet

# ----- Receiver side -----
def decrypt(self, esp_packet: dict) -> bytes:
    expected_icv = self._icv({k: v for k, v in esp_packet.items() if k != "icv"})
    if not hmac.compare_digest(expected_icv, esp_packet["icv"]):
        raise ValueError("ESP integrity check failed - packet rejected (possible tampering)")

    cipher = Cipher(algorithms.AES(self.enc_key), modes.CBC(esp_packet["iv"]))
    decryptor = cipher.decryptor()
    padded = decryptor.update(esp_packet["ciphertext"]) + decryptor.finalize()

    unpadder = padding.PKCS7(algorithms.AES.block_size).unpadder()
    plaintext = unpadder.update(padded) + unpadder.finalize()
    return plaintext

def _icv(self, fields: dict) -> bytes:
    mac = hmac.new(self.auth_key, digestmod=hashlib.sha256)
    mac.update(self.spi.to_bytes(4, "big"))
    mac.update(fields["seq_num"].to_bytes(4, "big"))
    mac.update(fields["iv"])
    mac.update(fields["ciphertext"])
    return mac.digest()

def demo():
    print("=" * 70)
    print("IPSec ESP (Encapsulating Security Payload) Simulation")
    print("=" * 70)

    enc_key = os.urandom(32) # AES-256 key
    auth_key = os.urandom(32) # HMAC key
    esp = ESP(spi=0x2002, enc_key=enc_key, auth_key=auth_key)

    payload = b"CONFIDENTIAL: transfer $10,000 to account 44521."
    print(f"\nOriginal payload : {payload}")

    packet = esp.encrypt(payload)
    print(f"ESP packet sent -> SPI={hex(packet['spi'])}, SeqNum={packet['seq_num']}")
    print(f" IV           : {packet['iv'].hex()}")
    print(f" Ciphertext   : {packet['ciphertext'].hex()[:60]}...")
    print(f" ICV          : {packet['icv'].hex()[:32]}...")

    print("\n[Case 1] Legitimate receiver decrypts payload:")
    recovered = esp.decrypt(packet)
    print(" Decrypted payload:", recovered)
    print(" Match original   :", recovered == payload)

    print("\n[Case 2] Attacker flips a bit in the ciphertext in transit:")
    tampered = dict(packet)
    corrupted_ct = bytearray(packet["ciphertext"])
    corrupted_ct[0] ^= 0xFF
    tampered["ciphertext"] = bytes(corrupted_ct)
    try:
        esp.decrypt(tampered)
    except ValueError as e:
        print(" Result:", e)

    print("\n[Case 3] Eavesdropper without the key only sees ciphertext:")
    print(" Wire data (no plaintext visible):", packet["ciphertext"].hex())

if __name__ == "__main__":
    demo()

```

Sample Output

```
=====
IPSec ESP (Encapsulating Security Payload) Simulation
=====
```

```
Original payload : b'CONFIDENTIAL: transfer $10,000 to account 44521.'
ESP packet sent -> SPI=0x2002, SeqNum=1
  IV      : ec4e0f0b9e6cd10f6c04b808fd25ca9e
  Ciphertext : 58f67e4cb654fd89cc5a31af7824da7ebb7f64bbf8c68d9141847f971c30...
  ICV      : 7a7172fda9a0e3b16c7d5a80f4e4849f...
```

```
[Case 1] Legitimate receiver decrypts payload:
  Decrypted payload: b'CONFIDENTIAL: transfer $10,000 to account 44521.'
  Match original   : True
```

```
[Case 2] Attacker flips a bit in the ciphertext in transit:
  Result: ESP integrity check failed - packet rejected (possible tampering)
```

```
[Case 3] Eavesdropper without the key only sees ciphertext:
  Wire data (no plaintext visible):
  58f67e4cb654fd89cc5a31af7824da7ebb7f64bbf8c68d9141847f971c3017cd884a4aecaa61c5fc20b1d6c359f5d5a6344799f0d51fd1a4d3c9df
```


3. Security Association (SA) Database Manager

Concept

A Security Association is a one-way (simplex) agreement between two IPSec endpoints that defines the security parameters — protocol (AH/ESP), mode (transport/tunnel), cryptographic algorithms, keys, and lifetime — used to protect a traffic flow. Per RFC 4301, an SA is uniquely identified by the triple (SPI, Destination IP Address, Security Protocol). Real IPSec stacks maintain this information in a Security Association Database (SAD), alongside a Security Policy Database (SPD) that decides which traffic needs an SA.

Design

- SecurityAssociation: stores SPI, endpoint, mode, algorithm names, keys, creation time, and lifetime; `is_expired()` implements soft SA-lifetime expiry so that keys are periodically refreshed (rekeying), reducing the exposure window of any single key.
- SADatabase: an in-memory dictionary keyed by (SPI, destination IP, protocol) offering `create_sa()`, `lookup()` (with an expiry warning), `delete_sa()`, `purge_expired()`, and `list_all()`.
- The demo creates two SAs (one with a short 2-second lifetime for demonstration), performs successful and failed lookups, waits for expiry, purges expired SAs, and explicitly deletes the remaining SA.

Python Implementation

```
"""
3. Security Association (SA) Database (SAD) Manager
-----
An IPSec Security Association is a simplex (one-way) logical connection
that defines the security parameters (algorithm, keys, mode, lifetime)
used to protect traffic between two endpoints. It is uniquely identified
by the triple (SPI, Destination IP, Protocol [AH/ESP]).

This program implements a small in-memory Security Association Database
(SAD) supporting create, lookup, list, and delete operations, plus
lifetime/expiry handling as required by RFC 4301.
"""

import os
import time
import uuid

class SecurityAssociation:
    def __init__(self, spi, dst_ip, protocol, mode, enc_algo, auth_algo,
                 enc_key, auth_key, lifetime_seconds=3600):
        self.spi = spi
        self.dst_ip = dst_ip
        self.protocol = protocol          # "AH" or "ESP"
        self.mode = mode                  # "transport" or "tunnel"
        self.enc_algo = enc_algo
        self.auth_algo = auth_algo
        self.enc_key = enc_key
        self.auth_key = auth_key
        self.created_at = time.time()
        self.lifetime_seconds = lifetime_seconds
        self.seq_num = 0

    def is_expired(self):
        return (time.time() - self.created_at) > self.lifetime_seconds

    def __repr__(self):
        status = "EXPIRED" if self.is_expired() else "ACTIVE"
        return (f"SA(SPI={hex(self.spi)}, dst={self.dst_ip}, proto={self.protocol}, "
                f"mode={self.mode}, enc={self.enc_algo}, auth={self.auth_algo}, "
                f"status={status})")
```

```

class SADatabase:
    """Security Association Database keyed by (SPI, dst_ip, protocol)."""

    def __init__(self):
        self._sad = {}

    @staticmethod
    def _key(spi, dst_ip, protocol):
        return (spi, dst_ip, protocol)

    def create_sa(self, dst_ip, protocol, mode="tunnel",
                  enc_algo="AES-256-CBC", auth_algo="HMAC-SHA256",
                  lifetime_seconds=3600):
        spi = int.from_bytes(os.urandom(4), "big")
        enc_key = os.urandom(32)
        auth_key = os.urandom(32)
        sa = SecurityAssociation(spi, dst_ip, protocol, mode, enc_algo, auth_algo,
                                enc_key, auth_key, lifetime_seconds)
        self._sad[self._key(spi, dst_ip, protocol)] = sa
        return sa

    def lookup(self, spi, dst_ip, protocol):
        sa = self._sad.get(self._key(spi, dst_ip, protocol))
        if sa is None:
            return None
        if sa.is_expired():
            print(f" [warning] SA {hex(spi)} has expired and should be renegotiated (rekeyed).")
        return sa

    def delete_sa(self, spi, dst_ip, protocol):
        key = self._key(spi, dst_ip, protocol)
        if key in self._sad:
            del self._sad[key]
            return True
        return False

    def purge_expired(self):
        expired = [k for k, sa in self._sad.items() if sa.is_expired()]
        for k in expired:
            del self._sad[k]
        return len(expired)

    def list_all(self):
        return list(self._sad.values())

def demo():
    print("=" * 70)
    print("IPSec Security Association (SA) Database Manager")
    print("=" * 70)

    sad = SADatabase()

    print("\n[1] Creating Security Associations:")
    sa1 = sad.create_sa(dst_ip="203.0.113.5", protocol="ESP", mode="tunnel",
                        lifetime_seconds=2)
    sa2 = sad.create_sa(dst_ip="198.51.100.9", protocol="AH", mode="transport",
                        lifetime_seconds=3600)
    print(" ", sa1)
    print(" ", sa2)

    print("\n[2] Lookup an existing SA:")
    found = sad.lookup(sa2.spi, sa2.dst_ip, sa2.protocol)
    print(" Found:", found)

    print("\n[3] Lookup a non-existent SA (wrong SPI):")
    not_found = sad.lookup(0xDEADBEEF, sa2.dst_ip, sa2.protocol)
    print(" Found:", not_found)

    print("\n[4] Waiting for SA #1 to expire (lifetime=2s)...")
    time.sleep(2.2)
    sad.lookup(sa1.spi, sa1.dst_ip, sa1.protocol)

```

```

print("\n[5] Purging expired SAs:")
removed = sad.purge_expired()
print(f"  Removed {removed} expired SA(s).")

print("\n[6] Deleting SA #2 explicitly:")
ok = sad.delete_sa(sa2.spi, sa2.dst_ip, sa2.protocol)
print("  Deletion successful:", ok)

print("\n[7] Remaining SAs in database:")
remaining = sad.list_all()
print(" ", remaining if remaining else " (empty)")

if __name__ == "__main__":
    demo()

```

Sample Output

```

=====
IPSec Security Association (SA) Database Manager
=====

[1] Creating Security Associations:
  SA(SPI=0x8ee64200, dst=203.0.113.5, proto=ESP, mode=tunnel, enc=AES-256-CBC, auth=HMAC-SHA256, status=ACTIVE)
  SA(SPI=0x995f95a7, dst=198.51.100.9, proto=AH, mode=transport, enc=AES-256-CBC, auth=HMAC-SHA256,
status=ACTIVE)

[2] Lookup an existing SA:
  Found: SA(SPI=0x995f95a7, dst=198.51.100.9, proto=AH, mode=transport, enc=AES-256-CBC, auth=HMAC-SHA256,
status=ACTIVE)

[3] Lookup a non-existent SA (wrong SPI):
  Found: None

[4] Waiting for SA #1 to expire (lifetime=2s)...
  [warning] SA 0x8ee64200 has expired and should be renegotiated (rekeyed).

[5] Purging expired SAs:
  Removed 1 expired SA(s).

[6] Deleting SA #2 explicitly:
  Deletion successful: True

[7] Remaining SAs in database:
  (empty)

```

4. Simplified PGP for Secure Email

Concept

PGP secures email using a hybrid cryptosystem: the message itself is encrypted with a fast symmetric cipher under a random, single-use session key, and only that small session key is encrypted with the recipient's RSA public key. Separately, the sender signs the plaintext with their own RSA private key so the recipient can verify both the sender's identity (authentication) and that the message was not altered (integrity), and the sender cannot later deny having sent it (non-repudiation).

Design

- PGPUser: generates an RSA-2048 keypair per user (their personal PGP keyring) and can export the public key (as would be published to a keyserver).
- `encrypt_and_sign()`: generates a random AES-256 session key, encrypts the message with AES-GCM (which provides built-in integrity via an authentication tag), encrypts the session key with the recipient's RSA public key using OAEP padding, and signs the original plaintext with the sender's RSA private key using PSS padding.
- `decrypt_and_verify()`: the recipient decrypts the session key with their private key, decrypts the message, then verifies the digital signature against the sender's public key.
- The demo shows a full round trip between Alice and Bob, a tampering attack that AES-GCM's authentication tag detects, and an impersonation attempt by 'Mallory' that is correctly rejected because her signature does not match Alice's public key.

Python Implementation

```
"""
4. Simplified PGP (Pretty Good Privacy) for Secure Email
-----
Real PGP combines:
  * Confidentiality -> hybrid encryption: a random session key encrypts
                        the message (symmetric, fast), and the session
                        key itself is encrypted with the recipient's
                        RSA public key (asymmetric).
  * Authentication  -> the sender signs a hash of the message with
                        their own RSA private key (digital signature).
  * Compression/Radix64 are omitted here for simplicity.

This program implements that same hybrid encrypt + sign flow using
RSA-2048 and AES-256-GCM via the `cryptography` library, simulating
Alice sending a signed & encrypted email to Bob.
"""

import os
from cryptography.hazmat.primitives.asymmetric import rsa, padding as asym_padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

class PGPUser:
    """Represents a mail user with an RSA keypair (their PGP keyring)."""

    def __init__(self, name):
        self.name = name
        self.private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        self.public_key = self.private_key.public_key()

    def export_public_key(self):
        """What you'd publish to a keyserver / attach to an email."""
        return self.public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo,
        )
```

```

class SimplifiedPGP:
    @staticmethod
    def encrypt_and_sign(sender: PGPUUser, recipient_public_key, message: bytes) -> dict:
        # 1) Generate a random one-time session key & encrypt the message (confidentiality)
        session_key = os.urandom(32)          # AES-256
        nonce = os.urandom(12)
        aesgcm = AESGCM(session_key)
        ciphertext = aesgcm.encrypt(nonce, message, associated_data=None)

        # 2) Encrypt the session key with the RECIPIENT's public RSA key
        encrypted_session_key = recipient_public_key.encrypt(
            session_key,
            asym_padding.OAEP(
                mgf=asym_padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None,
            ),
        )

        # 3) Sign a hash of the ORIGINAL message with the SENDER's private RSA key
        # (data-origin authentication + non-repudiation)
        signature = sender.private_key.sign(
            message,
            asym_padding.PSS(
                mgf=asym_padding.MGF1(hashes.SHA256()),
                salt_length=asym_padding.PSS.MAX_LENGTH,
            ),
            hashes.SHA256(),
        )

        return {
            "encrypted_session_key": encrypted_session_key,
            "nonce": nonce,
            "ciphertext": ciphertext,
            "signature": signature,
            "sender_name": sender.name,
        }

    @staticmethod
    def decrypt_and_verify(recipient: PGPUUser, sender_public_key, pgp_message: dict) -> bytes:
        # 1) Recipient decrypts the session key with THEIR private key
        session_key = recipient.private_key.decrypt(
            pgp_message["encrypted_session_key"],
            asym_padding.OAEP(
                mgf=asym_padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None,
            ),
        )

        # 2) Decrypt the message with the recovered session key
        aesgcm = AESGCM(session_key)
        plaintext = aesgcm.decrypt(pgp_message["nonce"], pgp_message["ciphertext"], associated_data=None)

        # 3) Verify the sender's signature over the plaintext using the SENDER's public key
        try:
            sender_public_key.verify(
                pgp_message["signature"],
                plaintext,
                asym_padding.PSS(
                    mgf=asym_padding.MGF1(hashes.SHA256()),
                    salt_length=asym_padding.PSS.MAX_LENGTH,
                ),
                hashes.SHA256(),
            )
            signature_valid = True
        except Exception:
            signature_valid = False

        return plaintext, signature_valid

def demo():
    print("=" * 70)

```

```

print("Simplified PGP - Secure Email Communication")
print("=" * 70)

alice = PGPUUser("alice@example.com")
bob = PGPUUser("bob@example.com")
print(f"\nGenerated RSA-2048 keypairs for {alice.name} and {bob.name}.")

email_body = b"Hi Bob, the Q3 contract is approved. Please countersign and return. - Alice"
print(f"\nOriginal email: {email_body.decode()}")

print("\n[Alice] Encrypting with Bob's public key & signing with her own private key...")
pgp_msg = SimplifiedPGP.encrypt_and_sign(alice, bob.public_key, email_body)
print(f"  Encrypted session key : {pgp_msg['encrypted_session_key'].hex()[:40]}...")
print(f"  Ciphertext (on wire)   : {pgp_msg['ciphertext'].hex()[:60]}...")
print(f"  Digital signature      : {pgp_msg['signature'].hex()[:40]}...")

print("\n[Bob] Decrypting with his private key & verifying Alice's signature...")
plaintext, valid = SimplifiedPGP.decrypt_and_verify(bob, alice.public_key, pgp_msg)
print("  Decrypted message :", plaintext.decode())
print("  Signature valid?   :", valid)

print("\n[Attack] Eve intercepts and tampers with the ciphertext in transit:")
tampered_msg = dict(pgp_msg)
corrupted = bytearray(pgp_msg["ciphertext"])
corrupted[0] ^= 0xFF
tampered_msg["ciphertext"] = bytes(corrupted)
try:
    SimplifiedPGP.decrypt_and_verify(bob, alice.public_key, tampered_msg)
    print("  Decryption unexpectedly succeeded (should not happen).")
except Exception as e:
    print(f"  Decryption failed as expected (AES-GCM auth tag check): {type(e).__name__}")

print("\n[Attack] Mallory tries to impersonate Alice by re-signing with her own key:")
mallory = PGPUUser("mallory@evil.com")
fake_msg = SimplifiedPGP.encrypt_and_sign(mallory, bob.public_key, b"Wire $50,000 to account 6666.")
_, valid2 = SimplifiedPGP.decrypt_and_verify(bob, alice.public_key, fake_msg)
print("  Signature verified against Alice's public key:", valid2, "(correctly rejected)")

if __name__ == "__main__":
    demo()

```

Sample Output

```

=====
Simplified PGP - Secure Email Communication
=====

Generated RSA-2048 keypairs for alice@example.com and bob@example.com.

Original email: Hi Bob, the Q3 contract is approved. Please countersign and return. - Alice

[Alice] Encrypting with Bob's public key & signing with her own private key...
  Encrypted session key : 711f2261d99a1b6995d5a93fa64d0d46b55a6fe7...
  Ciphertext (on wire)  : b2b502f1b694993bf53c64e230be3db9330342cc4a7ae75baa1dc96deae...
  Digital signature     : 75c5a1ec26252d9f25b05f11bf7fe1a33c4ec792...

[Bob] Decrypting with his private key & verifying Alice's signature...
  Decrypted message : Hi Bob, the Q3 contract is approved. Please countersign and return. - Alice
  Signature valid?  : True

[Attack] Eve intercepts and tampers with the ciphertext in transit:
  Decryption failed as expected (AES-GCM auth tag check): InvalidTag

[Attack] Mallory tries to impersonate Alice by re-signing with her own key:
  Signature verified against Alice's public key: False (correctly rejected)

```

5. Email Spam Classification

Concept

Production mail filters blend several independent signals rather than relying on one technique. This program combines: (a) header analysis — checking SPF/DKIM authentication results, From/Reply-To domain mismatches, display-name brand impersonation, and abnormal recipient counts; and (b) content-based filtering — a Naive Bayes bag-of-words classifier trained on labelled sample emails, plus a rule-based scan for spam phrases, excessive capitalization/punctuation, multiple embedded links, and mentions of large sums of money. The final decision blends the Bayes probability with the normalized heuristic score.

Design

- `analyze_headers()`: scores SPF/DKIM failures, Reply-To/From domain mismatch, brand-impersonation in the display name, and abnormal recipient counts — all classic phishing/spam header signatures.
- `NaiveBayesSpamFilter`: a from-scratch multinomial Naive Bayes text classifier with Laplace (add-1) smoothing, trained on a small labelled spam/ham corpus.
- `rule_based_score()`: flags spam phrases ('click here', 'act now', 'verify your account', etc.), ALL-CAPS abuse, excessive subject-line punctuation, multiple URLs, and large dollar amounts.
- `SpamClassifier.classify()`: combines the Bayes probability (50%) with the normalized header+rule heuristic score (50%) into a final score, and reports every contributing reason for transparency/explainability.

Python Implementation

```
"""
5. Email Spam Classification
-----
Combines two complementary techniques used by real mail-filtering
systems:

    (a) Header analysis    - checks SPF/DKIM-style authentication
                           results, suspicious "Reply-To" mismatches,
                           excessive recipients, and mismatched
                           display-name/domain (a classic phishing tell).

    (b) Content filtering  - a simple Naive Bayes bag-of-words model
                           trained on labelled sample emails, plus a
                           rule-based scan for spammy keywords/patterns
                           (ALL CAPS, urgency phrases, money amounts,
                           excessive punctuation, suspicious links).

The two scores are combined into one final classification, similar
to how production filters blend multiple signals (SpamAssassin-style
weighted scoring).
"""

import re
import math
from collections import defaultdict

class Email:
    def __init__(self, headers: dict, subject: str, body: str):
        self.headers = headers          # e.g. From, Reply-To, To, SPF, DKIM
        self.subject = subject
        self.body = body

    @property
    def full_text(self):
        return f"{self.subject} {self.body}"

# -----
```

```

# (a) Header-based heuristics
# -----
def analyze_headers(email: Email) -> (float, list):
    score = 0.0
    reasons = []

    if email.headers.get("SPF", "pass").lower() == "fail":
        score += 3.0
        reasons.append("SPF authentication failed")

    if email.headers.get("DKIM", "pass").lower() == "fail":
        score += 3.0
        reasons.append("DKIM signature invalid")

    from_addr = email.headers.get("From", "")
    reply_to = email.headers.get("Reply-To", "")
    from_domain = from_addr.split("@")[-1].rstrip(">") if "@" in from_addr else ""
    reply_domain = reply_to.split("@")[-1].rstrip(">") if "@" in reply_to else ""
    if reply_domain and from_domain and reply_domain != from_domain:
        score += 2.0
        reasons.append(f"Reply-To domain ({reply_domain}) differs from From domain ({from_domain})")

    display_name = email.headers.get("DisplayName", "")
    trusted_brands = ["paypal", "microsoft", "apple", "amazon", "bank"]
    for brand in trusted_brands:
        if brand in display_name.lower() and brand not in from_domain.lower():
            score += 4.0
            reasons.append(f"Display name impersonates '{brand}' but sender domain is '{from_domain}'")
            break

    num_recipients = email.headers.get("RecipientCount", 1)
    if num_recipients > 50:
        score += 1.5
        reasons.append(f"Sent to unusually large number of recipients ({num_recipients})")

    if not from_domain or from_domain.count(".") == 0:
        score += 1.0
        reasons.append("Malformed / missing sender domain")

    return score, reasons

# -----
# (b) Content-based Naive Bayes classifier
# -----
class NaiveBayesSpamFilter:
    def __init__(self):
        self.word_counts = {"spam": defaultdict(int), "ham": defaultdict(int)}
        self.class_counts = {"spam": 0, "ham": 0}
        self.vocab = set()

    @staticmethod
    def tokenize(text):
        return re.findall(r"[a-z']+", text.lower())

    def train(self, samples):
        """samples: list of (text, label) where label is 'spam' or 'ham'."""
        for text, label in samples:
            self.class_counts[label] += 1
            for word in self.tokenize(text):
                self.word_counts[label][word] += 1
                self.vocab.add(word)

    def _word_prob(self, self, word, label):
        # Laplace (add-1) smoothing
        count = self.word_counts[label][word]
        total = sum(self.word_counts[label].values())
        return (count + 1) / (total + len(self.vocab))

    def predict_proba(self, text):
        words = self.tokenize(text)
        total_docs = sum(self.class_counts.values())
        log_prob = {}
        for label in ("spam", "ham"):

```



```

        log_prob[label] = math.log(self.class_counts[label] / total_docs)
    for word in words:
        if word in self.vocab:
            log_prob[label] += math.log(self._word_prob(word, label))

    # Convert log-probabilities to a normalized spam probability
    max_log = max(log_prob.values())
    exp_probs = {k: math.exp(v - max_log) for k, v in log_prob.items()}
    total = sum(exp_probs.values())
    return exp_probs["spam"] / total

# -----
# (c) Rule-based keyword / pattern scan
# -----
SPAM_PHRASES = [
    "click here", "act now", "limited time", "verify your account",
    "winner", "free money", "urgent", "wire transfer", "congratulations",
    "you have won", "risk free", "guarantee", "unsubscribe",
]
URL_PATTERN = re.compile(r"https?:\/\/[^\s]+")

def rule_based_score(email: Email) -> (float, list):
    score = 0.0
    reasons = []
    text_lower = email.full_text.lower()

    for phrase in SPAM_PHRASES:
        if phrase in text_lower:
            score += 1.0
            reasons.append(f"Contains spam phrase: '{phrase}'")

    caps_words = re.findall(r"\b[A-Z]{4,}\b", email.full_text)
    if len(caps_words) >= 3:
        score += 1.5
        reasons.append(f"Excessive capitalization ({len(caps_words)} ALL-CAPS words)")

    if email.subject.count("!") >= 2:
        score += 1.0
        reasons.append("Excessive punctuation in subject line")

    urls = URL_PATTERN.findall(email.body)
    if len(urls) >= 2:
        score += 1.0
        reasons.append(f"Multiple embedded links ({len(urls)})")

    if re.search(r"\$\s?\d{3,}", email.body):
        score += 0.5
        reasons.append("Mentions large sums of money")

    return score, reasons

# -----
# (d) Combined classifier
# -----
class SpamClassifier:
    def __init__(self, bayes_filter: NaiveBayesSpamFilter, threshold=0.5):
        self.bayes = bayes_filter
        self.threshold = threshold

    def classify(self, email: Email):
        header_score, header_reasons = analyze_headers(email)
        rule_score, rule_reasons = rule_based_score(email)
        bayes_prob = self.bayes.predict_proba(email.full_text)

        # Weighted blend: header + rule heuristics normalized to 0-1, combined with Bayes prob
        heuristic_score = min((header_score + rule_score) / 10.0, 1.0)
        final_score = 0.5 * bayes_prob + 0.5 * heuristic_score
        label = "SPAM" if final_score >= self.threshold else "LEGITIMATE"

```

```

        return {
            "label": label,
            "final_score": round(final_score, 3),
            "bayes_probability": round(bayes_prob, 3),
            "header_score": header_score,
            "rule_score": rule_score,
            "reasons": header_reasons + rule_reasons,
        }

TRAINING_DATA = [
    ("Congratulations you have won a free prize click here to claim now", "spam"),
    ("URGENT verify your account immediately or it will be suspended", "spam"),
    ("Limited time offer act now guarantee risk free winner", "spam"),
    ("You have won free money wire transfer required unsubscribe", "spam"),
    ("Meeting rescheduled to 3pm tomorrow in conference room B", "ham"),
    ("Please find attached the quarterly report for your review", "ham"),
    ("Can we sync up on the project timeline this week", "ham"),
    ("Thanks for the update, I will review the document tonight", "ham"),
    ("Reminder your dentist appointment is scheduled for Friday", "ham"),
    ("Invoice attached for services rendered in October", "ham"),
]

def demo():
    print("=" * 70)
    print("Email Spam Classification (Header Analysis + Content Filtering)")
    print("=" * 70)

    bayes = NaiveBayesSpamFilter()
    bayes.train(TRAINING_DATA)
    classifier = SpamClassifier(bayes)

    test_emails = [
        Email(
            headers={"From": "security@paypal-verify.com", "Reply-To": "reply@totally-legit.ru",
                    "DisplayName": "PayPal Security", "SPF": "fail", "DKIM": "fail",
                    "RecipientCount": 500},
            subject="URGENT!! Verify your account NOW!!",
            body="Dear customer, CLICK HERE http://bit.ly/xyz http://tinyurl.com/abc to verify your "
                "account or it will be suspended. You have won $5000 free money guarantee!"
        ),
        Email(
            headers={"From": "manager@company.com", "Reply-To": "manager@company.com",
                    "DisplayName": "Priya Manager", "SPF": "pass", "DKIM": "pass",
                    "RecipientCount": 3},
            subject="Project timeline sync",
            body="Hi team, can we sync up on the project timeline this week? "
                "Please find attached the quarterly report for your review."
        ),
    ]

    for i, email in enumerate(test_emails, 1):
        print(f"\n--- Test Email #{i} ---")
        print(f"From: {email.headers.get('From')} Subject: {email.subject}")
        result = classifier.classify(email)
        print(f"Classification : {result['label']} (score={result['final_score']})")
        print(f" Bayes content probability : {result['bayes_probability']}")
        print(f" Header heuristic score : {result['header_score']}")
        print(f" Rule-based score : {result['rule_score']}")
        print(" Reasons:")
        for r in result["reasons"]:
            print(" -", r)

if __name__ == "__main__":
    demo()

```

Sample Output

```

=====
Email Spam Classification (Header Analysis + Content Filtering)
=====

```

--- Test Email #1 ---

From: security@paypal-verify.com Subject: URGENT!! Verify your account NOW!!

Classification : SPAM (score=1.0)

Bayes content probability : 1.0

Header heuristic score : 13.5

Rule-based score : 10.0

Reasons:

- SPF authentication failed
- DKIM signature invalid
- Reply-To domain (totally-legit.ru) differs from From domain (paypal-verify.com)
- Display name impersonates 'paypal' but sender domain is 'paypal-verify.com'
- Sent to unusually large number of recipients (500)
- Contains spam phrase: 'click here'
- Contains spam phrase: 'verify your account'
- Contains spam phrase: 'free money'
- Contains spam phrase: 'urgent'
- Contains spam phrase: 'you have won'
- Contains spam phrase: 'guarantee'
- Excessive capitalization (3 ALL-CAPS words)
- Excessive punctuation in subject line
- Multiple embedded links (2)
- Mentions large sums of money

--- Test Email #2 ---

From: manager@company.com Subject: Project timeline sync

Classification : LEGITIMATE (score=0.0)

Bayes content probability : 0.0

Header heuristic score : 0.0

Rule-based score : 0.0

Reasons:

6. Security Best Practices: IP, Email, and Web Services

The programs above demonstrate the underlying mechanics of IPsec and PGP. The following practices summarize how these tools — plus S/MIME for email — should be deployed and operated in real systems.

6.1 IP-Layer Security (IPsec) Best Practices

- Prefer ESP with combined-mode/AEAD ciphers (e.g., AES-GCM) over AH alone, or use ESP with authentication so a single pass provides both confidentiality and integrity.
- Use IKEv2 for automated, authenticated key exchange and periodic rekeying instead of static pre-shared keys; enable Perfect Forward Secrecy (PFS) so a compromised long-term key cannot decrypt past sessions.
- Set conservative SA lifetimes (both time-based and byte-count-based) to limit the exposure window of any single session key, and always purge/rekey expired SAs, as demonstrated by the SA manager.
- Use Tunnel mode with a properly configured Security Policy Database (SPD) for site-to-site/VPN traffic, and Transport mode only for host-to-host protection where both endpoints are trusted.
- Disable legacy weak algorithms (DES, MD5, HMAC-MD5, Diffie-Hellman Group 1/2) and enforce a minimum of AES-256 and SHA-256/384 with 2048-bit-or-larger DH groups (prefer ECDH).
- Enable anti-replay windows (sequence-number checking) on every SA, and log/alert on repeated integrity failures, which usually indicate an active attack rather than a transient error.

6.2 Email Security Best Practices (PGP and S/MIME)

- Always use hybrid encryption (symmetric session key + asymmetric key-wrapping) as PGP does — never encrypt large messages directly with RSA, which is slow and has strict size limits.
- Sign-then-encrypt or encrypt-then-sign consistently per protocol convention (PGP conventionally signs-then-encrypts) to avoid surrogate/impersonation attacks where a signed plaintext is re-wrapped by an attacker.
- Protect private keys with a strong passphrase and, where possible, hardware-backed storage (smart card, HSM, TPM, or a hardware security key); never transmit or back up unencrypted private keys.
- Use S/MIME (X.509 certificates issued by a trusted CA) in enterprise environments where centralized certificate management and native mail-client integration (Outlook, Apple Mail) are preferred; use PGP's web-of-trust model for decentralized, individual-to-individual trust.
- Publish and rotate keys through a trusted keyserver or your organization's certificate authority; publish key-revocation certificates in advance so a compromised key can be revoked quickly.
- Verify key fingerprints out-of-band (phone call, in person, QR code) before trusting a new correspondent's public key, to prevent man-in-the-middle key-substitution attacks.
- Deploy SPF, DKIM, and DMARC at the domain level in addition to PGP/S-MIME — they authenticate the transport/sending domain, while PGP/S-MIME authenticate the message content and sender identity end-to-end.
- Combine cryptographic protection with the spam/phishing classifier techniques shown in Section 5 (SPF/DKIM header checks, domain-mismatch detection, content scanning) as a defense-in-depth layer, since encryption alone does not stop phishing from a compromised or spoofed account.

6.3 Web Service Security Best Practices

- Enforce TLS 1.2+ (prefer TLS 1.3) for all web-service traffic, with HSTS enabled and legacy protocols (SSLv3, TLS 1.0/1.1) disabled.
- Use mutual TLS (mTLS) or IPsec tunnels for service-to-service traffic between backend microservices and across trust boundaries (e.g., between data centers or cloud VPCs).
- Apply the principle of least privilege to API keys and service credentials, rotate them regularly, and store secrets in a dedicated secrets manager rather than in code or configuration files.

- Validate and sanitize all input at the service boundary, rate-limit and monitor authentication endpoints, and log security-relevant events (failed integrity checks, signature failures, auth failures) for anomaly detection — mirroring the tamper-detection patterns demonstrated in the AH/ESP/PGP programs above.

Conclusion

Together, IPSec (AH + ESP, managed through Security Associations) secures data in transit at the network layer, while PGP/S-MIME secure email content end-to-end at the application layer, and layered content/header analysis (spam and phishing filtering) adds a behavioral defense that pure cryptography cannot provide. A well-architected system layers all of these controls rather than relying on any single mechanism.